Transcript-to-ISL Gloss

Hybrid Translation Architecture & Implementation Brief

Purpose

This document is an implementation brief for the AI coding tool building the project. It specifies how cleaned English transcripts should be converted into Indian Sign Language (ISL) gloss using a sequential hybrid architecture: an LLM translation pass followed by a mandatory rule-based pass and explicit validation.

Scope boundary

Build only the transcript-to-ISL-gloss component. Do not implement ASR/audio transcription, YouTube ingestion, 3D avatar rendering, sign-video retrieval, or video compositing as part of this task. Input is already-cleaned transcript text; downstream sign generation is a separate future phase.

Architecture decision

Yap & Render is the reference for the LLM translation layer. VirtualISLInterpreter is the primary reference for the rule-based secondary layer. Do not use Yap & Render's offline rule engine merely as a fallback; the rule-based pass must run after the LLM on every translation request.

Reference repositories

LLM translation reference: https://github.com/mitarthpathak/yap-render

Rule-based reference: https://github.com/krishnshyam/VirtualISLInterpreter

Prepared: 12 October 2026

1. Objective and non-negotiable decisions

Implement a maintainable, testable translation module that takes cleaned English transcript text and returns candidate/final ISL gloss with traceability. The architecture must be sequential—not a fallback chain.

- **Layer 1 — LLM:** generate a candidate ISL gloss from the English sentence or transcript chunk. Use the LLM translation implementation and prompting approach in Yap & Render as the starting reference.
- **Layer 2 — rule-based pass:** always process the LLM candidate. Study and adapt grammar/NLP logic from VirtualISLInterpreter. Do not assume its rules can be applied directly to gloss tokens without changes.
- **Layer 3 — validation:** run deterministic checks, identify unknown or questionable transformations, and attach review flags. Validation does not prove linguistic correctness.
- **Output:** return the original input, LLM candidate, post-rule output, validation findings, and processing metadata.
- **Human review:** plan for evaluation by an ISL-fluent reviewer. Do not label output production-accurate until independently evaluated.

2. Why this hybrid design

LLMs can interpret context and produce a candidate translation, but can also invent, omit, or inconsistently order glosses. Rules can make known transformations consistent and detect known issues, but a small collection of rules cannot encode all ISL grammar, discourse, non-manual markers, spatial grammar, or regional variation. The two layers should be complementary, with a transparent record of every change.

Important linguistic constraint: ISL gloss is an annotation convention, not simply English words with articles removed or words rearranged. A gloss sequence alone may not capture facial expression, head movement, spatial reference, or other non-manual/grammatical information. Avoid presenting unreviewed examples as authoritative ISL translations.

3. Reference repositories and how to use them

A. Yap & Render — LLM translation layer

Repository: https://github.com/mitarthpathak/yap-render

- Inspect the current repository, especially the translation API route and code that sends text to Gemini. The README identifies the Gemini translation path under app/api/translate-isl.
- Use this implementation as a reference for calling the LLM, constructing translation instructions, handling model responses, and separating translation from UI/avatar code.
- Extract only reusable translation logic. Do not copy the avatar player, speech-recognition input, UI, animation vocabulary, or unrelated application code into this translation-only module.
- Adapt the prompt and response contract for our use case: cleaned English transcript → candidate ISL gloss. Request structured output and prohibit explanatory prose mixed into the gloss field.
- Inspect repository licensing and attribution requirements before copying code. Prefer a small, clearly isolated adapter over copying the whole app.

B. VirtualISLInterpreter — mandatory rule-based second layer

Repository: https://github.com/krishnshyam/VirtualISLInterpreter

- Inspect its English-to-ISL gloss/NLP rules and supporting dependencies. Identify which operations transform English syntax and which are vocabulary mappings or preprocessing.
- Treat it as a source of candidate rules and implementation ideas—not as a guaranteed complete or validated ISL grammar engine.
- Design an adapter so the rule layer can accept the LLM candidate plus the original English sentence and relevant analysis. Some rules may need the source sentence; do not force every rule to operate on gloss-only input.

- Do not blindly apply rules that remove words, reorder tokens, or lemmatize glosses. Each rule must have a documented condition, intended effect, and tests.
- Inspect its license, dependencies, maintenance status, and compatibility before reusing code. If a rule is not appropriate for post-processing LLM output, implement a new, narrowly scoped rule instead.

C. What about Yap & Render's offline rule-based fallback?

Yap & Render documents its offline rule-based engine as the path used when the Gemini API key is absent or the API is unreachable. That does not establish that the engine is designed to consume and refine an LLM-generated gloss. It may be designed as an alternative translator from source text. Therefore, do not use it as the default second-pass engine without inspecting its input/output contract and tests. The chosen reference for the second pass is VirtualISLInterpreter; selected Yap rules may be evaluated later if they prove suitable, but they are not required.

4. Required end-to-end flow

1. Receive input

Accept cleaned English transcript text and optional metadata such as source ID, chunk ID, and timestamps.

2. Segment

Process manageable sentence/utterance chunks while preserving order and source offsets. Do not split blindly in a way that loses context.

3. LLM candidate

Send the source text and translation instructions to the LLM. Request candidate ISL gloss plus optional notes/uncertainty in separate structured fields.

4. Rule-based secondary pass

Always run the rule engine on every candidate. Supply the original source text and candidate gloss where useful. Apply only rules whose preconditions are satisfied.

5. Validate

Check schema, empty output, token format, known vocabulary/unsupported tokens, accidental prose, suspicious deletions, and whether rules changed output. These checks are diagnostics, not proof of ISL correctness.

6. Return traceable result

Return original source, candidate gloss, final gloss, applied rule IDs, warnings, model metadata, and a review status.

Architecture sketch

```
Cleaned English transcript
        |
        v
Chunk / sentence preparation
        |
        v
LLM translation (Yap & Render reference)
        |
        v
Candidate ISL gloss
        |
        v
MANDATORY rule-based second pass
(VirtualISLInterpreter reference; adapted)
        |
        v
Deterministic validation + uncertainty flags
        |
        v
Final gloss + audit trail + review status
```

5. Transcript cleaning and ASR artefact policy

Transcript cleaning is upstream of this module. The translator should receive cleaned text, but must still handle uncertainty conservatively if markers remain. Since this component receives text rather than audio, it cannot recover words misrecognized by ASR unless audio or a trusted correction is separately provided.

- **Obvious non-linguistic labels:** markers such as [whistle] or [cough] may be excluded from the translation text while being retained as source annotations when needed.
- **Fillers:** remove purely hesitant “uh/um” during upstream cleaning when appropriate. Do not automatically delete “hmm”, “oh”, “wow”, or similar interjections; context may make them meaningful.
- **Unclear speech:** preserve [inaudible] or an uncertainty flag rather than inventing missing words.
- **Traceability:** never overwrite the raw transcript. Store raw and cleaned text separately, along with cleaning actions.

6. Suggested data contract

Use a structured result similar to the following. Field names can be adapted to the existing project, but preserve the semantic separation.

```
{
  "source_text": "cleaned English sentence",
  "candidate_gloss": "LLM-generated candidate",
  "final_gloss": "post-rule gloss",
  "applied_rules": [
    {
      "rule_id": "RULE_ID",
      "description": "what changed",
      "status": "applied"
    }
  ],
  "validation": {
    "status": "pass | review_required | failed",
    "warnings": []
  },
  "metadata": {
    "model": "provider/model identifier",
    "prompt_version": "version",
    "rule_set_version": "version"
  }
}
```

Do not store chain-of-thought or hidden model reasoning. Store only the requested gloss, concise explanations of applied transformations where useful, validation findings, and operational metadata.

7. Rule-engine design requirements

- Rules must be modular, named, versioned, and independently testable.
- Every rule must define its input assumptions, preconditions, output behavior, and known limitations.
- Prefer conservative transformations. If confidence in a transformation is low, preserve the candidate and add a review flag.
- Avoid double-applying transformations. The rule pass should be deterministic for the same input and rule-set version.
- Keep lexical substitutions separate from syntactic transformations and from validation checks.
- Log changes as before/after spans or token sequences so reviewers can identify what the rule pass altered.
- Do not treat a generic English dependency parse or a simplistic SOV reorder as a complete model of ISL grammar.

8. Error handling and operational behavior

- If the LLM request fails, return an explicit translation failure or retry status. Do not silently substitute a different translator and label it as the normal hybrid output.

- If the LLM output is malformed, attempt a bounded repair/parse step or flag the result; do not silently invent missing gloss.
- The rule pass should still run on valid candidate outputs even when validation later finds warnings.
- If the rule pass errors, preserve the LLM candidate and return a failed/review-required status. Do not claim the secondary pass completed.
- Keep API keys in environment variables and out of source control and logs.
- For long transcripts, process bounded chunks with overlap/context where appropriate, preserve chunk order, and record chunk boundaries.

9. Evaluation plan and acceptance criteria

Before scaling to long transcripts, build a small evaluation set of approximately 30–50 representative sentences, including simple statements, questions, negation, time references, names, idioms, fillers/interjections, and ambiguous cases. Have an ISL-fluent reviewer assess outputs. Do not use raw word overlap as the sole quality metric.

- The LLM candidate and post-rule final output are both retained for comparison.
- The rule-based pass is invoked for every valid candidate, not only when the LLM fails.
- Every applied transformation is traceable to a rule ID or clearly documented transformation.
- Unknown words and uncertain translations are flagged instead of being silently replaced with invented gloss.
- Tests cover cases where rules should apply, should not apply, and should defer to human review.
- Evaluation compares LLM-only output with post-rule output using reviewer judgments for meaning preservation, grammatical acceptability, omissions, additions, and consistency.
- The system labels results as unreviewed until they have passed the agreed review process.

10. Implementation sequence for the coding agent

1. Inspect the existing project structure and identify the cleanest place for a standalone transcript-to-gloss module.
2. Inspect Yap & Render's translation API and isolate the LLM call/prompt/response handling. Record relevant source files in implementation notes.
3. Inspect VirtualISLInterpreter's code and document candidate reusable rules, dependencies, license, and limitations.
4. Implement the structured translation contract and the LLM candidate stage first.
5. Implement the mandatory rule-pass interface and add only well-understood rules with unit tests. Keep uncertain or linguistically risky rules disabled until reviewed.
6. Implement validation, warnings, traceability, and error states.
7. Create the 30–50 sentence evaluation set and compare LLM-only versus post-rule outputs.
8. Document which rules were reused, adapted, newly implemented, or intentionally excluded, with repository paths and license notes.

11. Explicit exclusions

- No speech-to-text or audio processing.
- No YouTube download, transcript ingestion, or video synchronization implementation.
- No 3D avatar, sign animation, fingerspelling playback, sign-video lookup, or video overlay.
- No claim of full ISL fluency or linguistic validation based only on repository output.
- No model fine-tuning until dataset suitability and evaluation needs are established.

12. Source references

1. Yap & Render repository: https://github.com/mitarthpathak/yap-render
Use as reference for the LLM translation call and response handling. The repository README describes Gemini translation and an offline rule-based fallback; inspect current source before adapting.

2. VirtualISLInterpreter repository: https://github.com/krishnshyam/VirtualISLInterpreter
Use as the primary reference for the mandatory rule-based secondary pass. Inspect current source, dependencies, license, and linguistic limitations before reuse.

Final instruction to the coding agent: Implement a modular, sequential hybrid translator. Use Yap & Render as the reference for the LLM layer and VirtualISLInterpreter as the reference for the mandatory rule-based second pass. Inspect and adapt both repositories rather than copying them wholesale. Preserve the candidate and final outputs, apply only justified transformations, flag uncertainty, and keep all code limited to transcript-to-ISL-gloss translation.